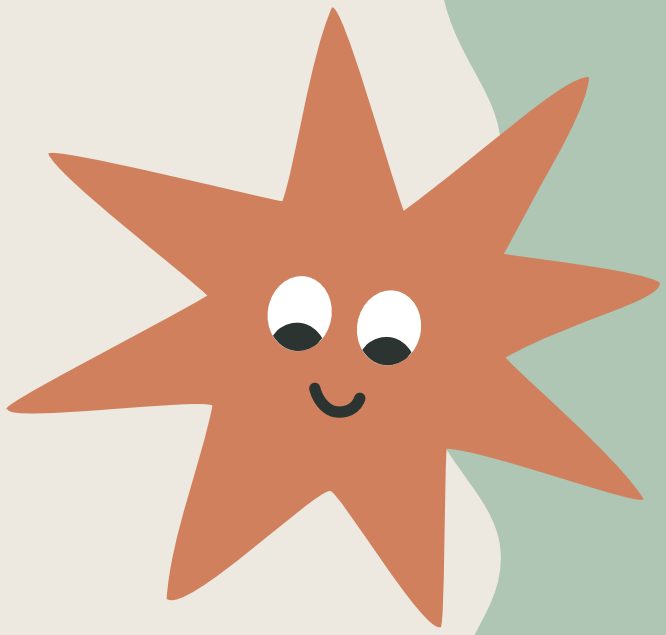
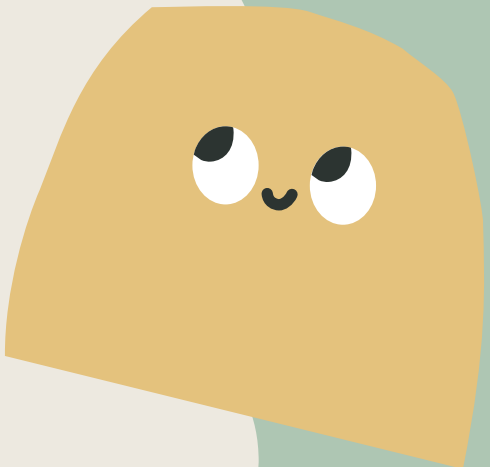
Introduction to Search in AI: Search Problems and State Space

Lecture 8



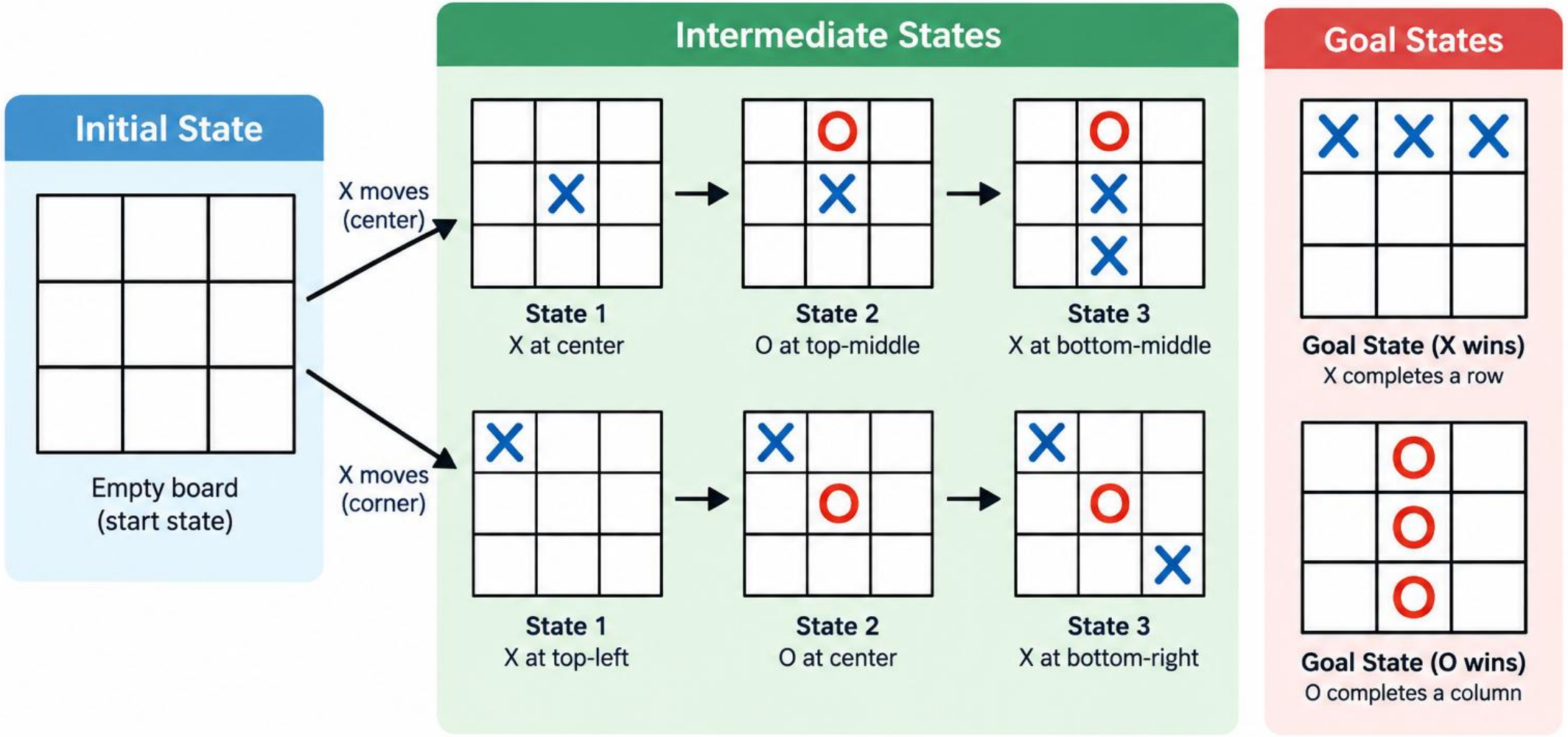


What Is Search in AI?

- In AI, search is the process of exploring possible **states** and **actions** to find a **path** from an **initial state** to a **goal state**
- In simple words
- Search means exploring possible solutions until a solution is found
- For example:
  - Suppose we want to travel from: Lahore → Islamabad
  - There may be several possible routes
  - An AI system can explore these possibilities and find a route that reaches Islamabad
- Why Does AI Need Search?

Prepared By: Ehtisham Rasheed

Tic-Tac-Toe: State Space Search Example



In state space search, each board configuration is a **state**, and moves (placing X or O) generate new states until a goal state is reached.

Search as Problem Solving

➢We can think of an AI search problem as:

Initial State

↓

Possible Actions

↓

New States

↓

More Actions

↓

Goal

Prepared By: Ehtisham Rasheed

What Is a Search Problem?

- A **search problem** describes:
- Where we start, what actions are possible, what happens after each action, and what condition represents success
- A typical search problem contains:
  1. Initial state
  2. Actions
  3. Transition model
  4. Goal test
  5. Path cost

Prepared By: Ehtisham Rasheed

What Is a Search Problem?

1. Initial State
   - The **initial state** is the state where the problem begins.
2. Actions
   - An **action** is an operation that can be performed from a state
3. Transition Model
   - The **transition model** describes the result of performing an action
   - It tells us what new state we reach after performing an action
4. Goal Test
   - The **goal test** determines whether the current state is the desired solution

Prepared By: Ehtisham Rasheed

What Is a Search Problem?

5. Path Cost
   - When a problem has multiple solutions, we may need to know how expensive each solution is?
   - We represent the path cost as: **g(n)**
   - Where: g(n) = cost of the path from the initial state to node n
   - Path cost may represent:
     - Distance
     - Time
     - Money
     - Number of actions
     - Energy consumption
   - Depending on the problem

Prepared By: Ehtisham Rasheed

What Is a State?

- A **state** represents a particular situation or configuration of the problem
- For a route-finding problem a state can be:
  - Lahore or Sialkot
- For a robot a state can be:
  - Robot at position A
- For 8-Puzzle a state can be:
  - Current arrangements of tiles
- For chess a state can be:
  - Current arrangement of all pieces on the board

Prepared By: Ehtisham Rasheed

What is State Space?

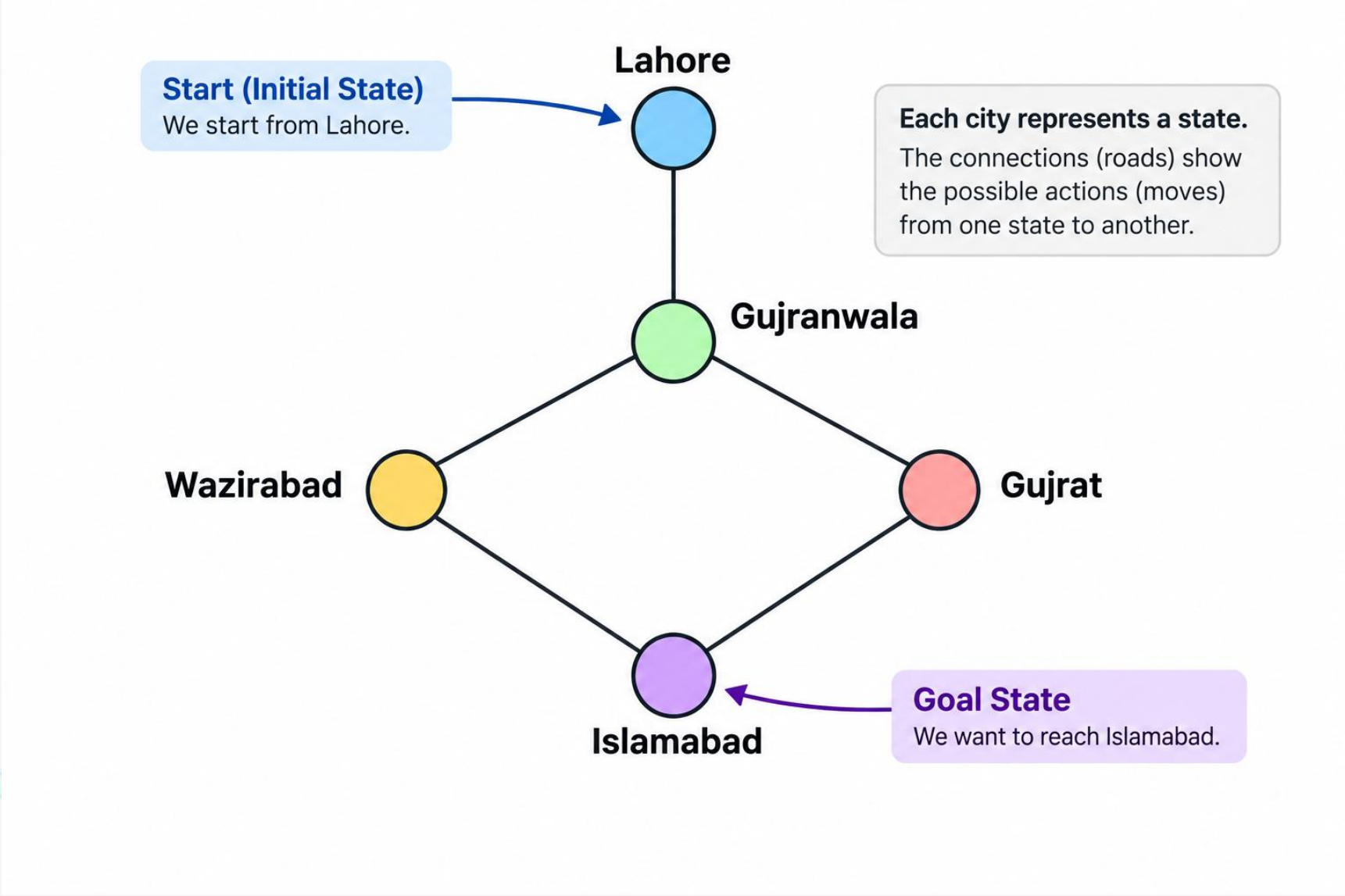
- The **state space** is the collection of all possible states that can be reached while solving a problem
- For example:



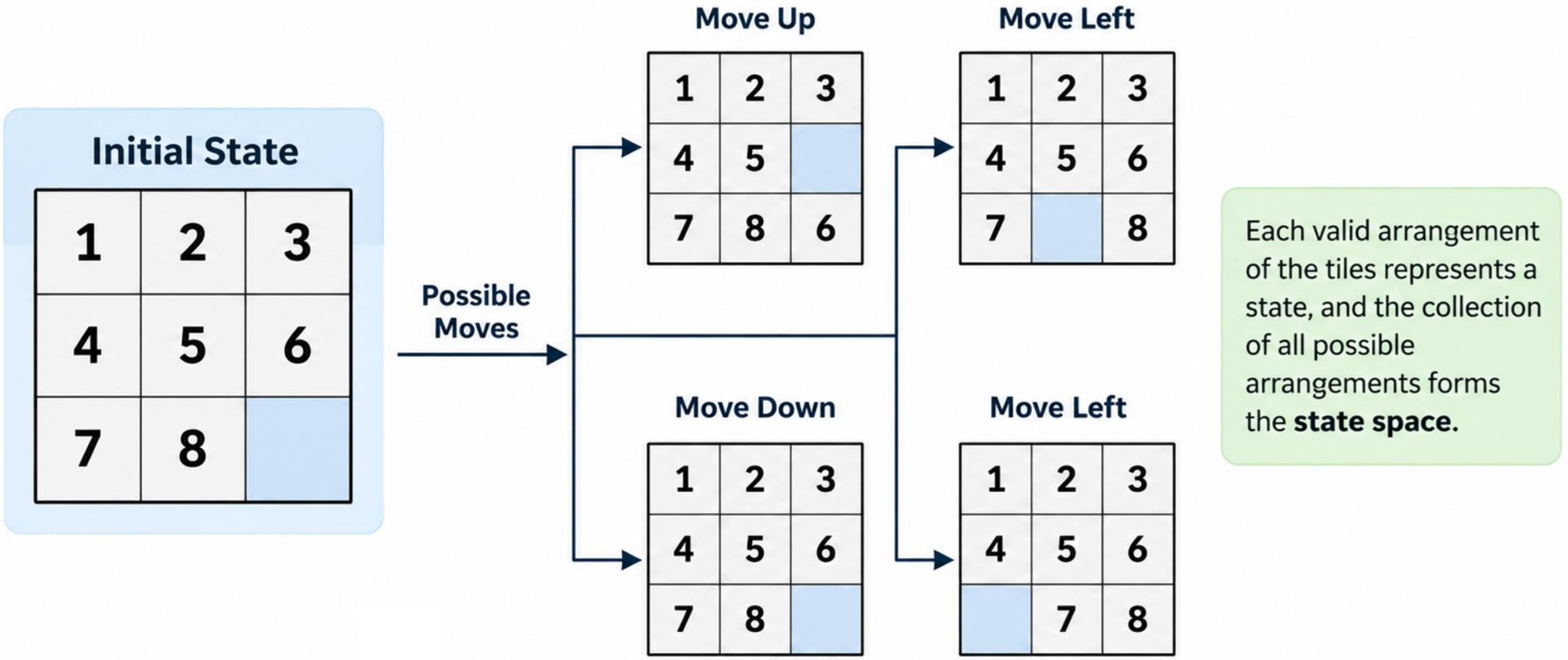
- Here: **A, B, C, D, E, F, G** represent possible states.
- Together, they form part of the problem's **state space**

Prepared By: Ehtisham Rasheed

State Space Example — Route Finding



State Space Example — 8-Puzzle



State Space Can Become Huge

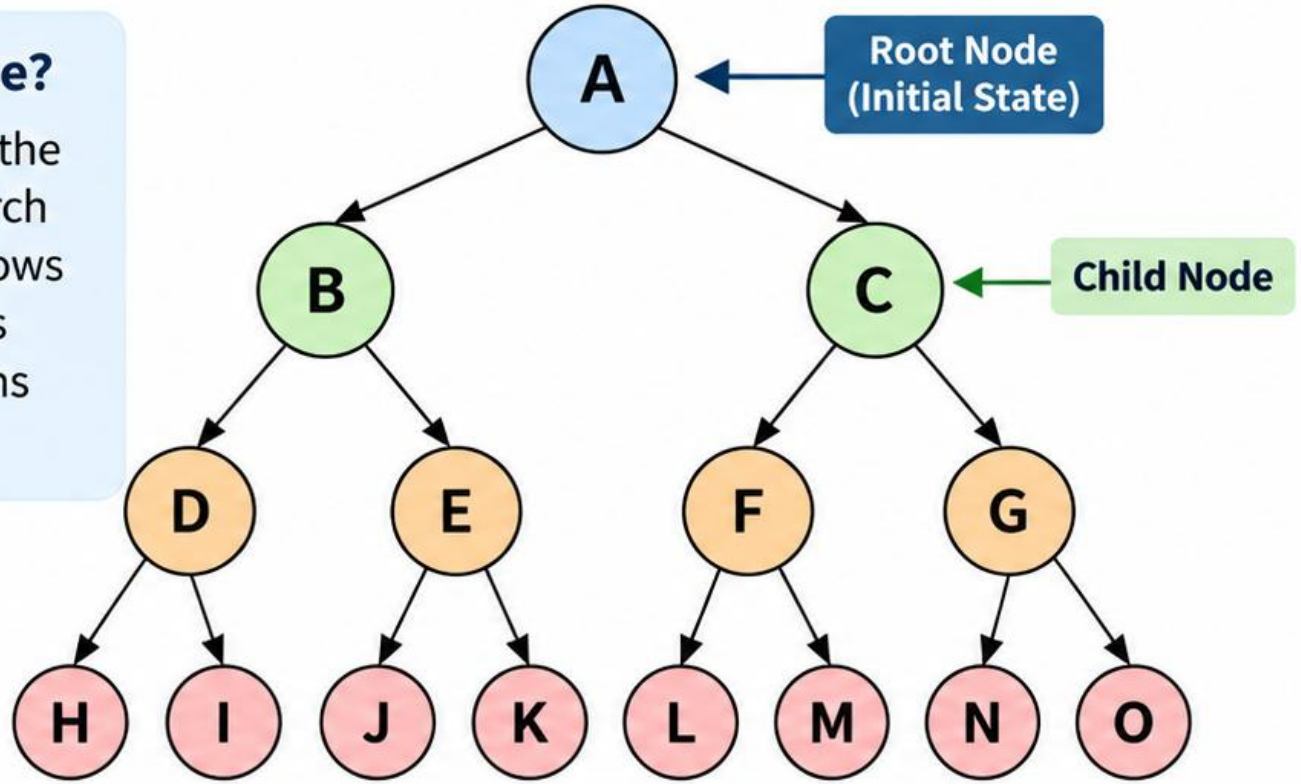
- One of the biggest challenges in AI search is the size of the state space
- A simple problem may have only a few states
- But a complex problem can have millions or billions of possible states
- For example:
  - Chess
  - Large route networks
  - Complex scheduling
  - Robotics
  - Large puzzles
- Therefore efficient search strategies are essential for solving large problems

Prepared By: Ehtisham Rasheed

Search Tree

What is a Search Tree?

A search tree represents the states explored by a search algorithm. Each node shows a state, and the branches represent possible actions leading to new states.



Key Terminology

A — Root Node
Represents the initial state of the problem.

B — Parent Node
A node that has one or more child nodes.

D — Child Node
A node generated from a parent node.

H — Leaf Node
A node with no children (terminal node or not yet expanded).

Example Explanation

- A is the root node (initial state).
- B and C are children of A.
- D and E are children of B, while F and G are children of C.
- H, I, J, K, L, M, N, and O are leaf nodes.

Important Points

- Each node represents a state in the problem.
- Each branch represents an action.
- The search tree shows how the algorithm explores the state space.
- The same state can appear multiple times in a search tree through different paths.

Path, Solution, Depth and Branching Factor

1. Path

A path is a sequence of nodes connected by edges from one node to another.

Example Path:

A → B → E → J

2. Solution

A solution is a path from the initial state (root) to a goal state.

Example Solution (if J is the goal):

A → B → E → J

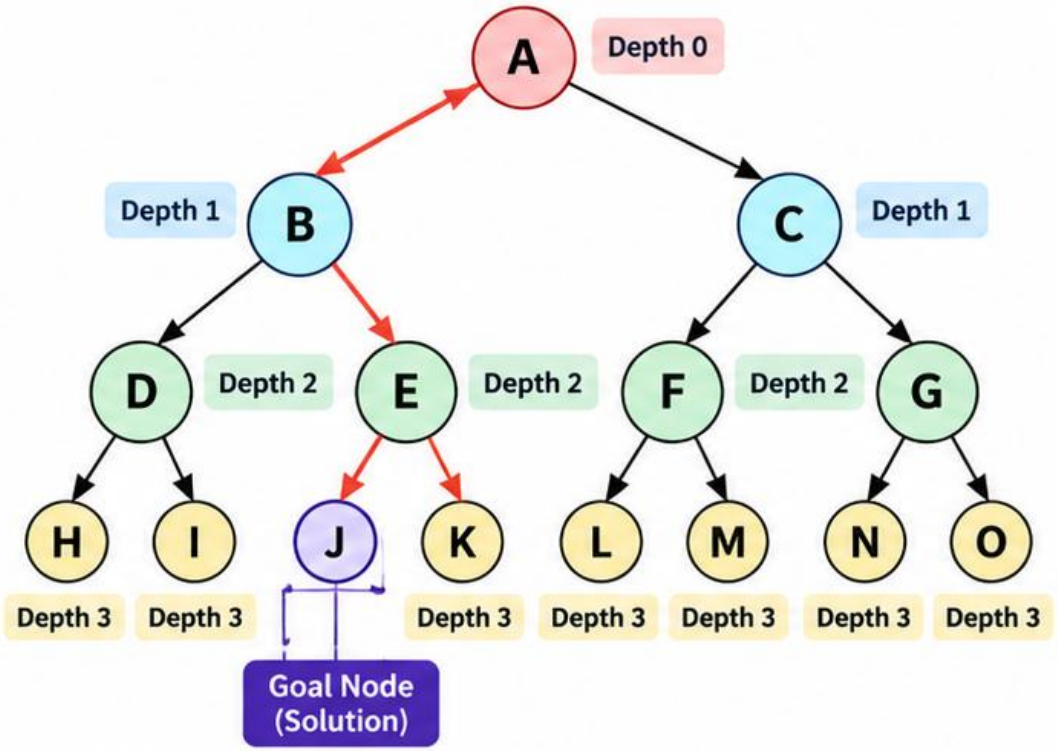
3. Depth

The depth of a node is the number of edges from the root node to that node.

Examples:

- Depth(A) = 0
- Depth(B) = 1
- Depth(E) = 2
- Depth(J) = 3

Search Tree Example



4. Branching Factor

The branching factor is the number of children (successors) a node can have.

Examples:

- Node **A** has 2 children → branching factor = **2**
- Node **B** has 2 children → branching factor = **2**
- Node **D** has 2 children → branching factor = **2**
- In this tree, the branching factor **(b) = 2**

Legend

- Root node (initial state)
- Depth 1 nodes
- Depth 2 nodes
- Depth 3 nodes (leaf nodes)
- Goal node
- Solution path

Key Takeaways

- A path is any sequence of nodes from one node to another.
- A solution is a path from the initial state to a goal state.
- Depth shows how far a node is from the root.
- Branching factor indicates how many children a node can have.
- In this example, the solution path is **A → B → E → J and the depth** of the goal node **J** is 3.

Branching Factor, Frontier and Explored Set

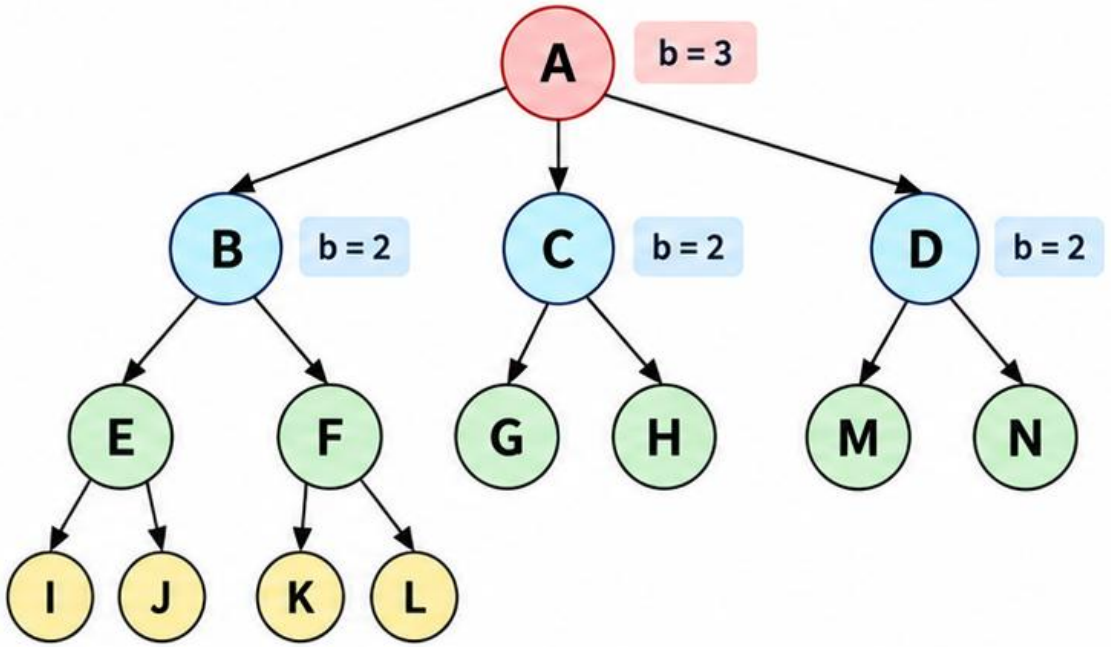
1. Branching Factor

The branching factor **(b)** is the number of children (successors) a node can have.

Example:

- Node A has 3 children (B, C, D), so its branching factor is 3.
- Node B has 2 children **(E, F)**, so its branching factor is 2.
- In this tree, the maximum branching factor **b = 3** (at node **A)**.

Search Tree Example



Step-by-Step Illustration

Initial State	After Expanding A	Next Step (Expand B)
Frontier: { A }	Frontier: { B, C, D }	Frontier: { C, D, E, F }
Explored: { }	Explored: { A }	Explored: { A, B }

2. Frontier

The frontier is the collection of generated nodes that are waiting to be explored.

Example:

- After expanding node A, the frontier contains: **{ B, C, D }**
- These nodes are generated but not yet explored.

3. Explored Set

The explored set contains the nodes that have already been explored (expanded).

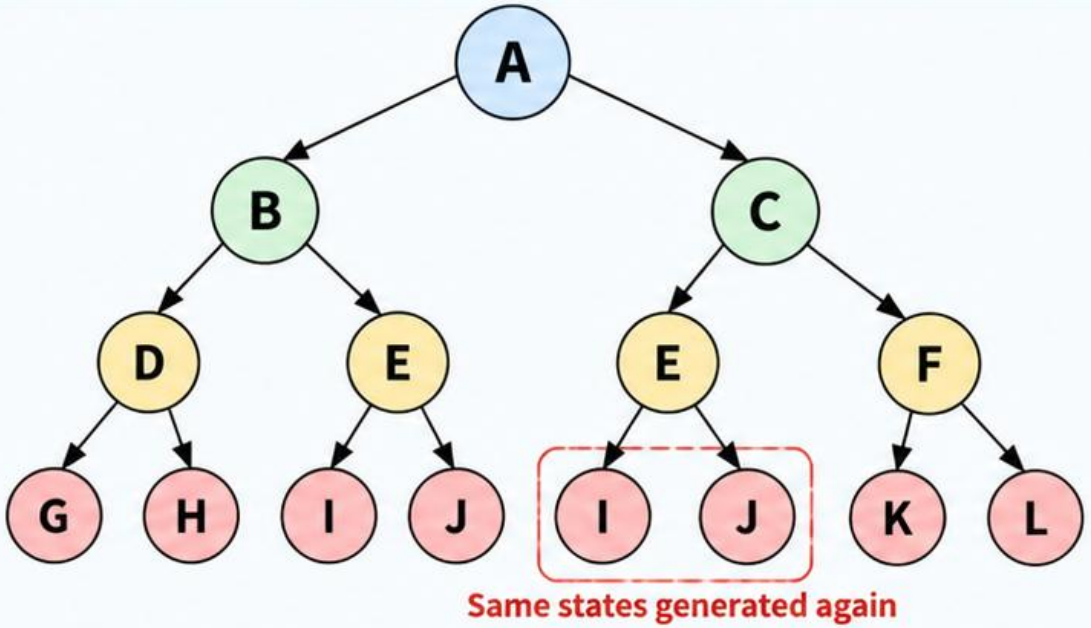
Example:

- After expanding node A, the explored set contains: **{ A }**
- These nodes will not be explored again (in graph search).

Tree Search vs Graph Search

Tree Search

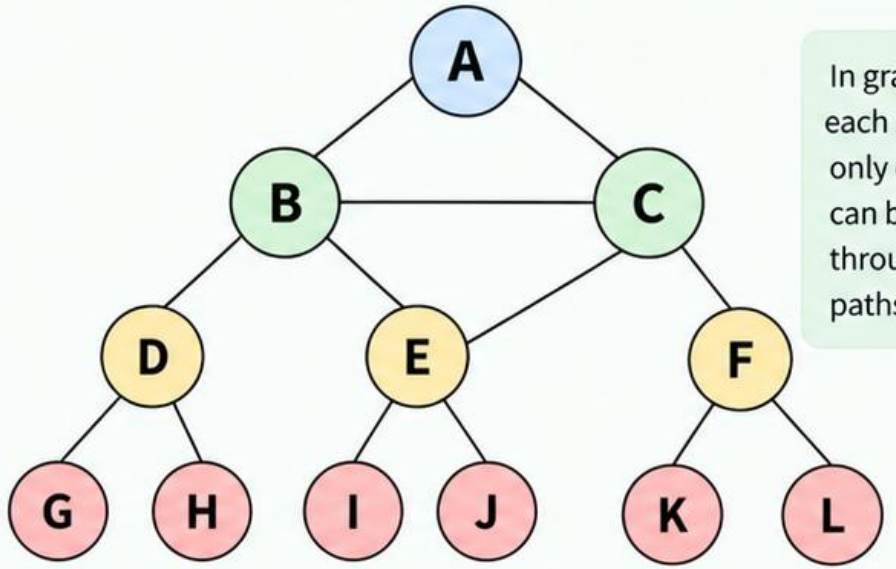
- Treats the search space as a tree.
- Does not check if a state has been visited before.
- The same state can be generated multiple times through different paths.
- Simpler, but may explore many repeated states.



Key Point:
In tree search, the same state (**I** and **J**) appears multiple times because there is no check for previously visited states.

Graph Search

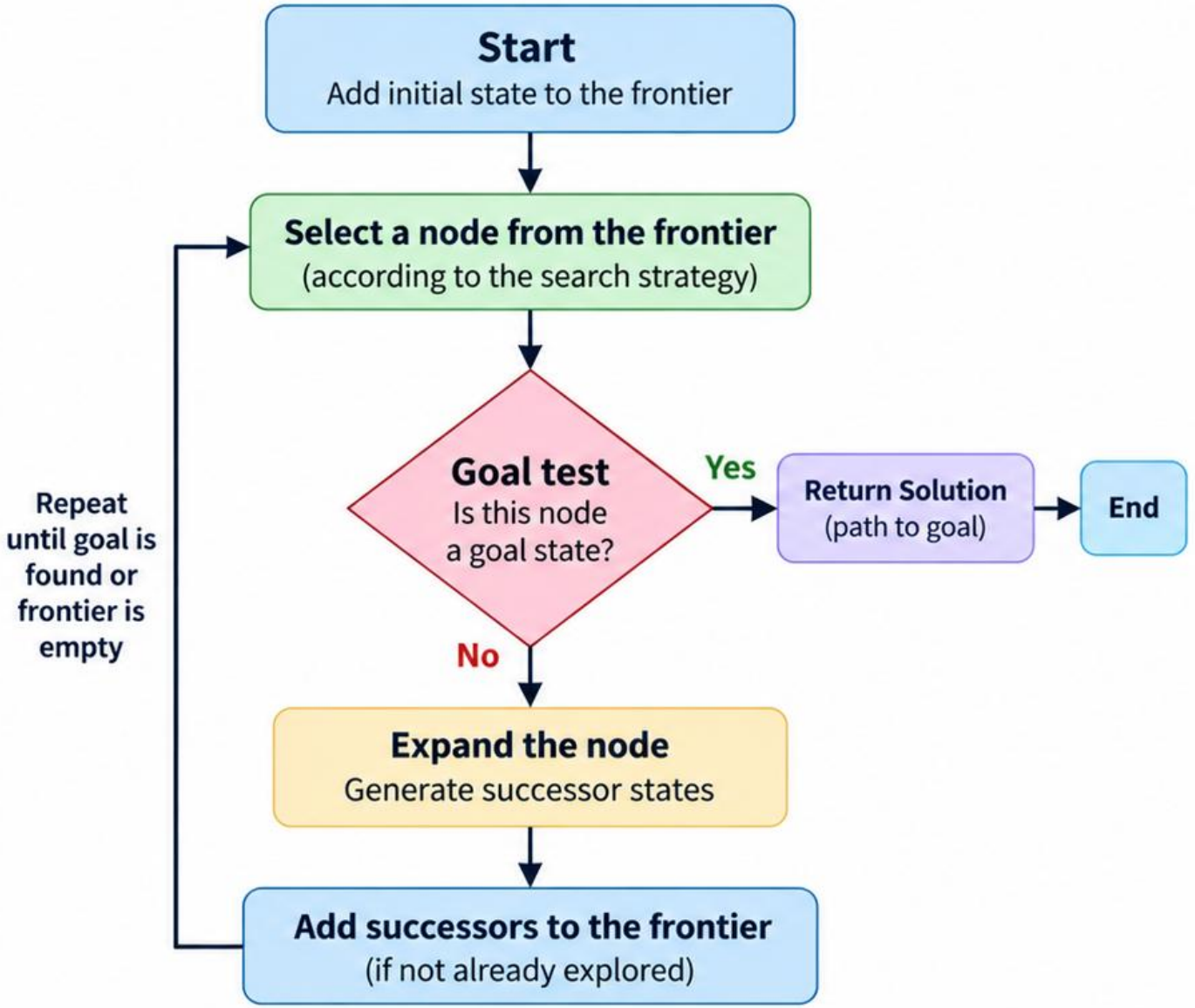
- Treats the search space as a graph.
- Keeps track of already visited (explored) states.
- Does not expand the same state again.
- Avoids repeated exploration and handles cycles.
- Usually more efficient than tree search.



In graph search, each state is explored only once, even if it can be reached through different paths.

Key Point:
In graph search, repeated states are avoided by checking the explored set, so each state (**I and J**) appears only once.

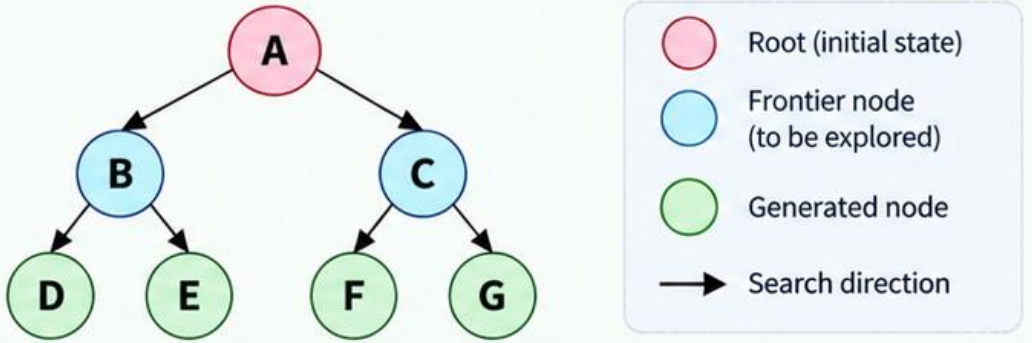
General Search Process



Steps in the General Search Process

1. Start with the initial state and add it to the frontier.
2. Select a node from the frontier (based on the search strategy).
3. Check if the selected node is a goal state.
4. If it is not a goal, expand the node (generate successor states).
5. Add the successors to the frontier (if not already explored).
6. Repeat the process until a goal is found or the frontier becomes empty.

Example Illustration



Key Takeaway

The general search process provides a framework. The only difference between search algorithms is how they choose the next node from the frontier.

Search in AI

Classification of Search Techniques

Classical Search

Searches for a solution by systematically exploring the search space. The problem is fully defined and a single-agent environment is assumed.

Uninformed Search

Uses no domain-specific knowledge (heuristics). Explores the search space based only on the problem definition.

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Iterative Deepening Search (IDS)

Informed Search

Uses domain-specific knowledge (heuristics) to guide the search toward a goal.

- Greedy Best-First Search
- A* Search

Local Search

Works with a single current state and moves to neighboring states to find a better solution. It does not maintain a complete search tree and is often used for optimization problems.

- Hill Climbing
- Beam Search
- Simulated Annealing
- Genetic Algorithms (evolutionary search)

Adversarial Search

Used in multi-agent environments where there are competing agents with conflicting goals (e.g., games). It considers the actions of an opponent while searching for the best move.

- Minimax Search
- Alpha-Beta Pruning
- Expectimax Search

Key Takeaway: Search in AI can be broadly classified into Classical Search, Local Search, and Adversarial Search. Uninformed and Informed Search are types of Classical Search.

Four Important Questions About Search Algorithms

➢Whenever we study a search algorithm, we should ask four questions

1. Is it Complete?
   - Will it find a solution if one exists?
2. Is it Optimal?
   - Will it find the best/lowest-cost solution?
3. What is its Time Complexity?
   - How much computation is required?
4. What is its Space Complexity?
   - How much memory is required?

Prepared By: Ehtisham Rasheed

1. Uninformed Search (Blind Search)

- Uninformed search algorithms have **no clues or extra information** about how close a particular state is to the goal
- They only know the initial state, the available actions, the transition model, and a test to check if they have reached the goal
- They explore the state space blindly and systematically
- Examples:
  - Breadth-First Search
  - Depth-First Search
  - Iterative Deepening

Prepared By: Eltishama Rasmeed

2. Informed Search (Heuristic Search)

- Informed search algorithms use **domain-specific knowledge** to estimate how close a state is to the goal
- This estimation is called a **heuristic function, h(n)**
- It allows the algorithm to prioritize exploring paths that look the most promising
- Examples:
  - Greedy Best-First Search
  - A* Search

Prepared By: Ehtisham Rasheed